Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology

Department of Electrical & Computer Engineering

Course Title

Digital Techniques Sessional

Course No: ECE 2112 **Lab No:** 01

Report Title

Implementation of Logic Gates Using Universal Gates, Full Adder, and Binary to BCD Converter

Date of Submission: 26 July, 2026

Submitted To	Submitted By
MST. Mazeda Noor Tasnim Lecturer Department of Electrical & Computer Engineering, RUET	Nazmul Islam Nice Roll: 2410005 ECE-24 Series

Problem 1 — Implementation of an OR Gate Using Only NOR Gates

Objective

Design and verify an OR gate using two NOR gates, demonstrating that NOR gates alone can replicate other basic logic functions.

Theory

A NOR gate alone can build any other gate, which is why it is called a universal gate. The output of one NOR gate is already the inverse of OR (A+B). So if we send this signal into a second NOR gate with both its inputs tied together, it gets inverted once more and we get back the OR function:

$$G_1 = \overline{A+B}, \qquad Y = \overline{G_1 + G_1} = \overline{G_1} = A + B$$

If both inputs of a NOR gate are connected to the same signal X, its output simply becomes the complement of X. In other words, the gate now works like a plain inverter (NOT gate).

Truth Table

A	B	G_1	Y = A + B
0	0	1	0
0	1	0	1
1	0	0	1
1	1	0	1

Circuit Diagram

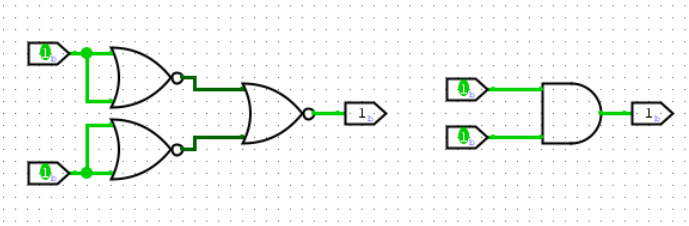

Figure 1: OR gate implemented using two NOR gates

Discussion

Since both inputs of the second NOR gate are connected to G_1, it works like an inverter and gives back A + B at the output. The truth table shows this circuit behaves exactly like a normal OR gate for every input.

Conclusion

An OR gate was successfully realized using two NOR gates, verified against its complete truth table.

Problem 2 — Implementation of an OR Gate Using Only NAND Gates

Objective

Design and verify an OR gate using three NAND gates by applying De Morgan's theorem.

Theory

By De Morgan's theorem:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

which is basically a NAND operation performed on Ā and B̄. Since a NAND gate with its inputs tied together works as an inverter, we use one NAND gate to get Ā, another to get B̄, and a third NAND gate to combine them:

$$\overline{A} = \overline{A \cdot A}, \qquad \overline{B} = \overline{B \cdot B}, \qquad Y = \overline{\overline{A} \cdot \overline{B}} = A + B$$

Truth Table

A	B	Ā	B̄	Y = A + B
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

Circuit Diagram

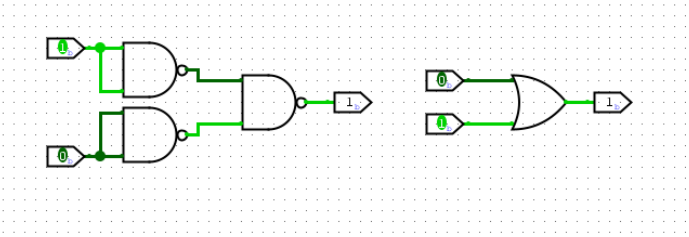

Figure 2: OR gate implemented using three NAND gates

Discussion

The first two NAND gates have their inputs shorted, so they just act as inverters and produce Ā and B̄. The third NAND gate combines these, which by De Morgan's theorem gives A + B. This design needs one more gate than the NOR version, because NAND is naturally suited to AND-type logic and needs an extra inversion step to produce OR.

Conclusion

An OR gate was successfully realized using three NAND gates, matching the expected truth table for all input combinations.

Problem 3 — Implementation of an AND Gate Using Only NOR Gates

Objective

Design and verify an AND gate using three NOR gates by applying De Morgan's theorem.

Theory

By De Morgan's theorem:

$$A \cdot B = \overline{\overline{A} + \overline{B}}$$

which is basically a NOR operation on Ā and B̄. Two NOR gates are used as inverters to get Ā and B̄, and a third NOR gate combines them:

$$\overline{A} = \overline{A + A}, \qquad \overline{B} = \overline{B + B}, \qquad Y = \overline{\overline{A} + \overline{B}} = A \cdot B$$

Truth Table

A	B	Ā	B̄	Y = A · B
0	0	1	1	0
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

Circuit Diagram

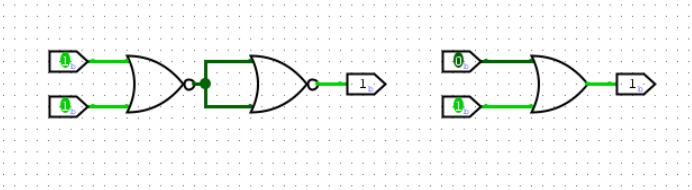

Figure 3: AND gate implemented using three NOR gates

Discussion

The first two NOR gates invert A and B to give Ā and B̄. The third NOR gate combines these, which gives the AND function by De Morgan's theorem. Just like the NAND-based OR gate, this needs one extra inversion stage because NOR is naturally suited to OR-type logic.

Conclusion

An AND gate was successfully realized using three NOR gates, with correct behaviour confirmed for all input combinations.

Problem 4 — Implementation of an AND Gate Using Only NAND Gates

Objective

Design and verify an AND gate using two NAND gates.

Theory

A NAND gate simply gives the complement of AND:

$$Y = \overline{A \cdot B}$$

If this output is fed into a second NAND gate whose inputs are tied together, it gets inverted again and we get back the AND function:

$$G_1 = \overline{A \cdot B}, \qquad Y = \overline{G_1 \cdot G_1} = \overline{G_1} = A \cdot B$$

Truth Table

A	B	G_1 = complement of A·B	Y = A · B
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Circuit Diagram

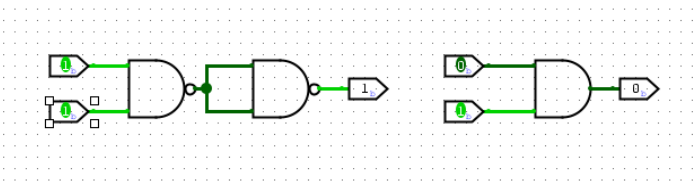

Figure 4: AND gate implemented using two NAND gates

Discussion

G_1, which is the complement of A·B, is fed into both inputs of the second NAND gate. This inverts it once more and gives back the AND relationship. Since NAND is a universal gate, only two gates are needed here, just like the NOR-based OR gate in Problem 1.

Conclusion

An AND gate was successfully realized using two NAND gates, matching the expected truth table.

Problem 5 — Implementation of a NOT Gate Using Only a NOR Gate

Objective

Design and verify a NOT gate using a single NOR gate.

Theory

The NOR operation is:

$$Y = \overline{A + B}$$

Tying both inputs to the same signal A reduces this to:

$$Y = \overline{A + A} = \overline{A}$$

which is exactly the NOT function.

Truth Table

A	Y = Ā
0	1
1	0

Circuit Diagram

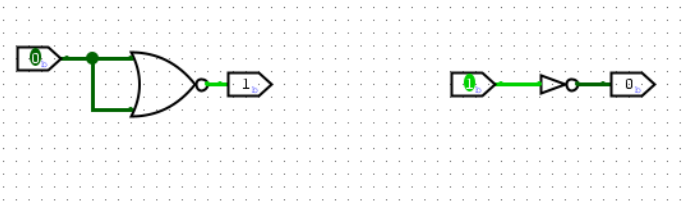

Figure 5: NOT gate implemented using a single NOR gate

Discussion

When both inputs of the NOR gate are shorted together, the OR part of its behaviour disappears and only the inversion remains. This is the simplest possible construction using a universal gate, and it is the same trick used in Problems 1 and 3.

Conclusion

A NOT gate was successfully realized using a single NOR gate with tied inputs.

Problem 6 — Implementation of a NOT Gate Using Only a NAND Gate

Objective

Design and verify a NOT gate using a single NAND gate.

Theory

The NAND operation is:

$$Y = \overline{A \cdot B}$$

Tying both inputs to the same signal A reduces this to:

$$Y = \overline{A \cdot A} = \overline{A}$$

which is exactly the NOT function.

Truth Table

A	Y = Ā
0	1
1	0

Circuit Diagram

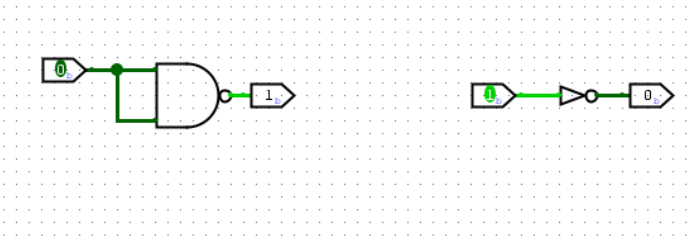

Figure 6: NOT gate implemented using a single NAND gate

Discussion

Just like the NOR-based inverter, shorting both NAND inputs removes the AND part of its behaviour and leaves only inversion. This shows that both NOR and NAND can, by themselves, be used to build every other basic logic gate.

Conclusion

A NOT gate was successfully realized using a single NAND gate with tied inputs.

Problem 7 — Implementation and Verification of a Full Adder Circuit

Objective

Design a full adder from basic logic gates and verify its Sum and Carry outputs for all eight input combinations of A, B, and Cin.

Theory

A full adder adds three single-bit inputs — A, B, and a carry-in Cin — and gives a Sum bit and a carry-out bit Cout. Unlike a half adder, it can also take an incoming carry, which is what lets several full adders be chained together to build multi-bit adders:

$$Sum = A \oplus B \oplus C_{in}, \qquad C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$$

It can be built using two half adders and one OR gate. The first half adder computes P = A⊕B and A·B. The second half adder combines P with Cin to give the final Sum, along with the term P·Cin. Finally, the OR gate combines A·B and P·Cin to produce Cout.

Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuit Diagram

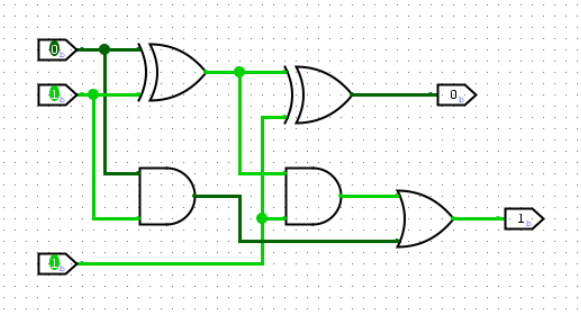

Figure 7: Full adder circuit using two XOR gates, two AND gates, and one OR gate

Discussion

The first XOR gate gives the partial sum P = A⊕B, and at the same time the first AND gate gives the direct carry A·B. The second XOR gate then combines P with Cin to produce the final Sum, while the second AND gate produces P·Cin. Since a carry can come from either path, the OR gate combines A·B and P·Cin to give Cout. All eight rows of the truth table matched what was expected.

Conclusion

A full adder was successfully implemented and verified against its complete truth table, confirming correct three-bit binary addition.

Problem 8 — Implementation of a Binary to BCD Converter

Objective

Design a combinational circuit that converts a 4-bit binary number (0–15) into its two-digit Binary Coded Decimal (BCD) representation.

Theory

A 4-bit binary number B3B2B1B0 can represent decimal values from 0 to 15. But BCD only allows digits 0–9, so values from 10 to 15 need two digits: a tens digit T (0 or 1) and a units digit U3U2U1U0 (0–9). If the binary value is 9 or less, T = 0 and the units digit is simply the input itself. If it is 10 or more, T = 1 and the units digit is the input minus 10.

Karnaugh map simplification of the 16-row truth table gives:

$$T = B_3(B_2 + B_1)$$

$$U_3 = B_3\overline{B_2}\,\overline{B_1}$$

$$U_2 = \overline{B_2}B_3 + B_2\overline{B_1}$$

$$U_1 = B_1\overline{B_3} + \overline{B_2}B_1\overline{B_3}$$

$$U_0 = B_0$$

U0 is always equal to B0. This is because 10 is an even number, so subtracting it never changes whether the last bit is 0 or 1.

Truth Table

B3	B2	B1	B0	T	U3	U2	U1	U0
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1
0	0	1	0	0	0	0	1	0
0	0	1	1	0	0	0	1	1
0	1	0	0	0	0	1	0	0
0	1	0	1	0	0	1	0	1

0	1	1	0	0	0	1	1	0
0	1	1	1	0	0	1	1	1
1	0	0	0	0	1	0	0	0
1	0	0	1	0	1	0	0	1
1	0	1	0	1	0	0	0	0
1	0	1	1	1	0	0	0	1
1	1	0	0	1	0	0	1	0
1	1	0	1	1	0	0	1	1
1	1	1	0	1	0	1	0	0
1	1	1	1	1	0	1	0	1

Circuit Diagram

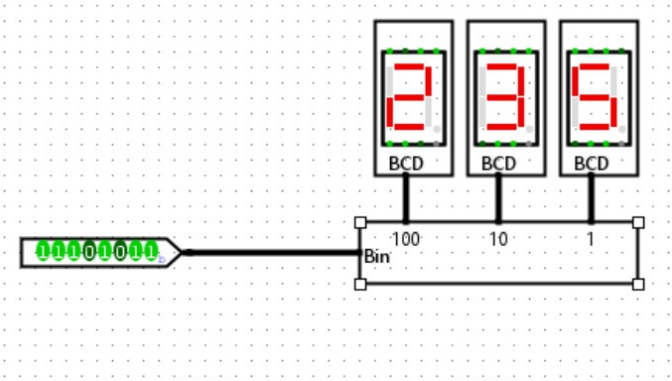

Figure 8: Binary to BCD converter driving three seven-segment BCD displays

Discussion

Each output was written as its own Boolean function of B3B2B1B0, after simplifying with a Karnaugh map. The tens bit T basically works like a threshold detector — it only turns high when the input is 10 or more. Whenever T = 1, the units bits effectively subtract 10, which is why their equations share terms with T. Because the input is limited to 4 bits, the whole circuit can be built directly with AND, OR, and NOT gates, without needing a separate adder-subtractor stage.

Conclusion

A binary to BCD converter for 4-bit inputs (0–15) was successfully designed using Karnaugh-map-derived combinational logic. The truth table confirmed correct separation into the tens bit T and units nibble U3U2U1U0 for every input value.

References

1. M. M. Mano, Digital Design, 5th ed. Upper Saddle River, N.J.: Pearson, 2013.

2. R. J. Tocci, N. S. Widmer, and G. L. Moss, Digital Systems: Principles and Applications, 11th ed. Upper Saddle River, N.J.: Pearson, 2011.